

ChatGPT

Exercícios gerados pelo ChatGPT a partir do pedido: “Crie 10 exercícios de algoritmo em Python (no nível intermediário, envolvendo engenharia mecânica, elétrica e telecomunicações), com enunciados no contexto das engenharias mencionadas, resoluções com pseudocódigo em linguagem algorítmica em português e resoluções com código Python, e uma lista com respostas no final das resoluções”.

1. Cálculo de Torque (Engenharia Mecânica)

Enunciado

Calcule o torque gerado por uma força aplicada perpendicularmente a uma alavanca.

Fórmula:

$$T = F \cdot d$$

Onde:

- F = força (N)
- d = distância (m)

Pseudocódigo

None

INÍCIO

ler força F

ler distância d

torque $\leftarrow F * d$

mostrar torque

FIM

Python

Python

```
F = float(input("Força (N): "))
```

```
d = float(input("Distância (m): "))
```

```
torque = F * d
```

```
print("Torque =", torque, "N·m")
```

2. Potência Elétrica

Enunciado

Calcule a potência elétrica consumida por um dispositivo.

$$P = V \cdot I$$

Pseudocódigo

None

INÍCIO

ler tensão V

ler corrente I

potência $\leftarrow V * I$

mostrar potência

FIM

Python

Python

```
V = float(input("Tensão (V): "))
I = float(input("Corrente (A): "))

P = V * I

print("Potência =", P, "W")
```

3. Velocidade Angular

 Enunciado

Calcule a velocidade angular de um eixo.

$$\omega = \frac{2 \times \pi \times n}{60}$$

Onde n = RPM

Pseudocódigo

None

INÍCIO

ler rotação n

$\omega \leftarrow (2 * \pi * n) / 60$

mostrar ω

FIM

Python

Python

```
import math
```

```

n = float(input("Rotação (RPM): "))

omega = (2 * math.pi * n) / 60

print("Velocidade angular =", omega, "rad/s")

```

4. Lei de Ohm

 Enunciado

Calcule a corrente em um circuito.

$$I = \frac{V}{R}$$

Pseudocódigo

None

INÍCIO

```

ler tensão V
ler resistência R
se R ≠ 0 então
    I ← V / R
    mostrar I
senão
    mostrar erro

```

FIM

Python

Python

```

V = float(input("Tensão (V): "))
R = float(input("Resistência (Ω): "))

if R != 0:
    I = V / R
    print("Corrente =", I, "A")
else:
    print("Erro: resistência não pode ser zero")

```

5. Atenuação de Sinal (Telecom)

 Enunciado

Calcule a atenuação em dB:

$$A = 10 \log_{10}(P_{\text{entrada}}/P_{\text{saida}})$$

Pseudocódigo

None

INÍCIO

ler Pentrada

ler Psaida

$A \leftarrow 10 * \log_{10}(\text{Pentrada} / \text{Psaida})$

mostrar A

FIM

Python

Python

```
import math
```

```
Pentrada = float(input("Potência de entrada: "))
```

```
Psaida = float(input("Potência de saída: "))
```

```
A = 10 * math.log10(Pentrada / Psaida)
```

```
print("Atenuação =", A, "dB")
```

6. Energia Cinética

 Enunciado

Calcule a energia cinética de um corpo.

$$E = \frac{m.v^2}{2}$$

Pseudocódigo

None

INÍCIO

ler massa m

ler velocidade v

$E \leftarrow 0.5 * m * v^2$

mostrar E

FIM

Python

Python

```
m = float(input("Massa (kg): "))
v = float(input("Velocidade (m/s): "))

E = 0.5 * m * v**2

print("Energia cinética =", E, "J")
```

7. Frequência de Onda

 Enunciado

Calcule a frequência de uma onda:

$$f = \frac{v}{\lambda}$$

Pseudocódigo

None

```
INÍCIO
  ler velocidade v
  ler comprimento λ
  f ← v / λ
  mostrar f
FIM
```

Python

Python

```
v = float(input("Velocidade da onda (m/s): "))
l = float(input("Comprimento de onda (m): "))

f = v / l

print("Frequência =", f, "Hz")
```

8. Rendimento de Máquina

 Enunciado

Calcule o rendimento:

$$n = \frac{P_{\text{útil}}}{P_{\text{total}}} \times 100$$

Pseudocódigo

None

INÍCIO

ler Putil

ler Ptotal

$\eta \leftarrow (P_{\text{util}} / P_{\text{total}}) * 100$

mostrar η

FIM

Python

Python

```
Putil = float(input("Potência útil: "))
```

```
Ptotal = float(input("Potência total: "))
```

```
eta = (Putil / Ptotal) * 100
```

```
print("Rendimento =", eta, "%")
```

9. Capacitância Equivalente (Série)

Enunciado

Calcule a capacitância equivalente:

$$C_{eq} = \frac{C_1 \cdot C_2}{C_1 + C_2}$$

Pseudocódigo

None

INÍCIO

ler C1, C2

$C_{eq} \leftarrow 1 / (1/C1 + 1/C2)$

mostrar Ceq

FIM

Python

Python

```
C1 = float(input("Capacitância C1: "))
```

```
C2 = float(input("Capacitância C2: "))
```

$$C_{eq} = 1 / (1/C1 + 1/C2)$$

```
print("Capacitância equivalente =", Ceq)
```

10. Taxa de Transmissão (Telecom)

 Enunciado

Calcule a taxa de transmissão:

$$R = \frac{dados}{tempo}$$

Pseudocódigo

None

INÍCIO

 ler dados

 ler tempo

$R \leftarrow \text{dados} / \text{tempo}$

 mostrar R

FIM

Python

Python

```
dados = float(input("Dados transmitidos (bits): "))
```

```
tempo = float(input("Tempo (s): "))
```

```
R = dados / tempo
```

```
print("Taxa =", R, "bps")
```

Gabarito (Resumo das Respostas)

1. Torque $\rightarrow T = F * d$

2. Potência $\rightarrow P = V * I$

3. Velocidade angular $\rightarrow \omega = (2\pi n)/60$
4. Corrente $\rightarrow I = V / R$
5. Atenuação $\rightarrow A = 10 \log_{10}(P_e/P_s)$
6. Energia cinética $\rightarrow E = 0.5 m v^2$
7. Frequência $\rightarrow f = v / \lambda$
8. Rendimento $\rightarrow \eta = (P_{\text{útil}} / P_{\text{total}}) * 100$
9. Capacitância equivalente $\rightarrow 1/(1/C_1 + 1/C_2)$
10. Taxa de transmissão $\rightarrow R = \text{dados} / \text{tempo}$

Gemini

Exercícios gerados pelo Gemini a partir do pedido: “Crie 10 exercícios de algoritmo em Python (no nível intermediário, envolvendo engenharia mecânica, elétrica e telecomunicações), com enunciados no contexto das engenharias mencionadas, resoluções com pseudocódigo em linguagem algorítmica em português e resoluções com código Python, e uma lista com respostas no final das resoluções”.

Exercício 1: Engenharia Mecânica - Análise de Fadiga de Materiais

Enunciado: Crie um programa que receba o limite de resistência à tração (S_u) e a tensão aplicada (σ) em um componente. O programa deve calcular o limite de resistência à fadiga estimado (S_e) usando a aproximação de $0.5 \times S_u$ e determinar se o componente falhará por fadiga (se $\sigma > S_e$).

- Pseudocódigo:

None

```
Algoritmo "AnaliseFadiga"
Var Su, Tensao, Se: Real
Inicio
    Escreva("Limite de tração (MPa): ")
    Leia(Su)
    Escreva("Tensão aplicada (MPa): ")
    Leia(Tensao)
    Se <- Su * 0.5
    Se Tensao > Se Entao
        Escreva("Falha por fadiga detectada.")
    Senao
        Escreva("Componente seguro.")
    FimSe
FimAlgoritmo
```

-
- Código Python:
Python

None

```
su = float(input("Limite de tração (MPa): "))
tensao = float(input("Tensão aplicada (MPa): "))
se = su * 0.5
status = "Falha" if tensao > se else "Seguro"
print(f"Limite Se: {se} MPa | Status: {status}")
```

-

Exercício 2: Engenharia Elétrica - Correção de Fator de Potência

Enunciado: Desenvolva um algoritmo que receba a Potência Ativa (P em kW) e o Fator de Potência atual (FP_{atual}). Calcule a Potência Reativa (Q) necessária para elevar o FP para 0.95. Considere $Q = P \times (\tan(\phi_{atual}) - \tan(\phi_{alvo}))$.

- Pseudocódigo:

None

```
Algoritmo "CorrecaoFP"
Var P, FP_atual, Ang_at, Ang_alvo, Q_nec: Real
Inicio
    Leia(P, FP_atual)
    Ang_at <- arc_cos(FP_atual)
```

```

    Ang_alvo <- arc_cos(0.95)
    Q_nec <- P * (tan(Ang_at) - tan(Ang_alvo))
    Escreva("Potência Reativa necessária (kVar): ", Q_nec)
FimAlgoritmo

```

-
- Código Python:
Python

None

```

import math
p = float(input("Potência Ativa (kW): "))
fp_atual = float(input("FP atual: "))
ang_atual = math.acos(fp_atual)
ang_alvo = math.acos(0.95)
q_necessaria = p * (math.tan(ang_atual) - math.tan(ang_alvo))
print(f"Capacitor necessário: {q_necessaria:.2f} kVar")

```

-

Exercício 3: Telecomunicações - Cálculo de Orçamento de Link (Link Budget)

Enunciado: Calcule a potência recebida (Pr) em um enlace de rádio. Dados: Potência de transmissão (Pt), Ganho da antena transmissora (Gt), Ganho da receptora (Gr) e Perda no percurso (L). Fórmula: $Pr = Pt + Gt + Gr - L$ (todos em dB).

- Pseudocódigo:

None

```

Algoritmo "LinkBudget"
Var Pt, Gt, Gr, L, Pr: Real
Inicio
    Leia(Pt, Gt, Gr, L)
    Pr <- Pt + Gt + Gr - L
    Escreva("Potência Recebida: ", Pr, " dBm")
FimAlgoritmo

```

-
- Código Python:
Python

None

```
pt = float(input("Potência Transmissão (dBm): "))
gt = float(input("Ganho Tx (dBi): "))
gr = float(input("Ganho Rx (dBi): "))
perda = float(input("Perda no percurso (dB): "))
pr = pt + gt + gr - perda
print(f"Sinal recebido: {pr} dBm")
```

-

Exercício 4: Engenharia Mecânica - Vazão em Tubulações

Enunciado: Calcule o Número de Reynolds (Re) para um fluido em uma tubulação. $Re = (\rho \times v \times D) / \mu$. Classifique como Laminar ($Re < 2000$), de Transição ($2000 \leq Re \leq 4000$) ou Turbulento ($Re > 4000$).

- **Código Python:**
Python

None

```
rho = float(input("Densidade (kg/m³): "))
v = float(input("Velocidade (m/s): "))
d = float(input("Diâmetro (m): "))
mu = float(input("Viscosidade (Pa.s): "))
re = (rho * v * d) / mu

if re < 2000: tipo = "Laminar"
elif re <= 4000: tipo = "Transição"
else: tipo = "Turbulento"
print(f"Reynolds: {re:.0f} - Fluxo {tipo}")
```

-

Exercício 5: Engenharia Elétrica - Análise de Malhas (Lei de Kirchhoff)

Enunciado: Dada uma malha simples com uma fonte V e três resistores em série (R1, R2, R3), calcule a corrente do circuito e a queda de tensão em R2.

- **Código Python:**
Python

None

```
v = float(input("Tensão da fonte (V): "))
```

```
resistores = [float(x) for x in input("R1 R2 R3 (ohms): ").split()]
req = sum(resistores)
corrente = v / req
vr2 = corrente * resistores[1]
print(f"Corrente: {corrente:.2f}A | V_R2: {vr2:.2f}V")
```

•

Exercício 6: Telecomunicações - Taxa de Erro de Bit (BER) Simulada

Enunciado: Crie uma função que receba o número de bits transmitidos e o número de bits recebidos com erro. Retorne a BER em notação científica.

- **Código Python:**
Python

None

```
bits = int(input("Total de bits: "))
erros = int(input("Bits com erro: "))
ber = erros / bits
print(f"BER: {ber:.2e}")
```

•

Exercício 7: Engenharia Mecânica - Torque em Eixos

Enunciado: Calcule o diâmetro mínimo necessário de um eixo sólido sujeito a um torque T e uma tensão de cisalhamento máxima permitida τ . Fórmula: $D = \sqrt[3]{(16 \times T) / (\pi \times \tau)}$

- **Código Python:**
Python

None

```
import math
t = float(input("Torque (N.m): "))
tau = float(input("Tensão adm (Pa): "))
d = ((16 * t) / (math.pi * tau))**(1/3)
print(f"Diâmetro mínimo: {d*1000:.2f} mm")
```

•

Exercício 8: Engenharia Elétrica - Sistema Trifásico

Enunciado: Em um sistema trifásico equilibrado ligado em estrela (Y), a tensão de linha VL é informada. Calcule a tensão de fase Vf e a potência total Pt dado uma corrente I e fator de potência FP. ($V_L = \sqrt{3} \times V_f$; $P_t = \sqrt{3} \times V_L \times I \times FP$)

- **Código Python:**
Python

None

```
v1 = float(input("Tensão de linha (V): "))
i = float(input("Corrente (A): "))
fp = float(input("FP: "))
vf = v1 / math.sqrt(3)
pt = math.sqrt(3) * v1 * i * fp
print(f"V_fase: {vf:.2f}V | P_total: {pt/1000:.2f} kW")
```

-

Exercício 9: Telecomunicações - Capacidade de Canal (Shannon)

Enunciado: Calcule a capacidade máxima de um canal (C) em bps, dada a largura de banda (B em Hz) e a relação sinal-ruído (SNR em dB). $C = B \times \log_2(1 + SNR_{linear})$. Note:

$$SNR_{linear} = 10^{(SNR_{dB}/10)}$$

- **Código Python:**
Python

None

```
b = float(input("Bandwidth (Hz): "))
snr_db = float(input("SNR (dB): "))
snr_linear = 10**(snr_db / 10)
c = b * math.log2(1 + snr_linear)
print(f"Capacidade: {c/1e6:.2f} Mbps")
```

-

Exercício 10: Engenharia Mecânica - Termodinâmica (Gás Ideal)

Enunciado: Crie um programa que resolva a equação de Clapeyron ($PV = nRT$). O usuário deve escolher qual variável deseja calcular (P, V ou T), fornecendo as outras. Considere $R = 8.314$.

- **Código Python:**
Python

None

```
calc = input("Calcular (P/V/T)? ").upper()
n = float(input("n (mols): "))
```

```

r = 8.314
if calc == 'P':
    v, t = float(input("V: ")), float(input("T: "))
    res = (n * r * t) / v
elif calc == 'V':
    p, t = float(input("P: ")), float(input("T: "))
    res = (n * r * t) / p
# ... (lógica similar para T)
print(f"Resultado {calc}: {res:.2f}")

```

Lista de Respostas (Gabarito para Testes)

1. Fadiga: Se $S_u=400$ e $\sigma=250 \rightarrow$ Falha (Limite 200).
2. Fator Potência: $P=100$, $FP=0.8 \rightarrow 42.14$ kVA.
3. Link Budget: $P_t=20$, $G_t=15$, $G_r=15$, $L=100 \rightarrow -50$ dBm.
4. Reynolds: $P=1000$, $v=2$, $D=0.05$, $\mu=0.001 \rightarrow 100.000$ (Turbulento).
5. Kirchhoff: $V=12$, $R=\{10, 20, 30\} \rightarrow I=0.2A$, $VR_2=4V$.
6. BER: $Bits=10^6$, $Erros=10 \rightarrow 1.00e-05$.
7. Torque: $T=100$, $\tau=40 \times 10^6 \rightarrow 23.35$ mm.
8. Trifásico: $V_L=380$, $I=10$, $FP=0.85 \rightarrow V_f=219.39V$, $P_t=5.59$ kW.
9. Shannon: $B=10^6$, $SNR=30dB \rightarrow 9.97$ Mbps.
10. Gás Ideal: $n=1$, $T=300$, $V=0.025 \rightarrow P \approx 99768$ Pa.

NotebookLM

Exercícios gerados pelo NotebookLM a partir do pedido: “Crie 10 exercícios de algoritmo em Python (no nível intermediário, envolvendo engenharia mecânica, elétrica e telecomunicações), com enunciados no contexto das engenharias mencionadas, resoluções com pseudocódigo em linguagem algorítmica em português e resoluções com código Python, e uma lista com respostas no final das resoluções” e envio três PDFs como referências (PDFs enviados pelo professor [Diego Luis Kreutz](#) no Classroom).

Exercício 1: Análise de Circuitos Resistivos (Elétrica)

Enunciado: Escreva um programa que receba uma lista de resistências de um circuito. O usuário deve escolher se a associação é em série ou em paralelo. O programa deve usar uma função para calcular a resistência equivalente (R_{eq}), a corrente total (I) para uma tensão V fornecida e a potência total dissipada (P).

Pseudocódigo:

None

ALGORITMO Calculadora_Circuito

INÍCIO

LEIA n_resistores

```

PARA i DE 1 ATÉ n_resistores:
    LEIA r
    ADICIONE r À lista_resistores
LEIA tipo_associacao ("serie" OU "paralelo")
LEIA tensao_v
SE tipo_associacao == "serie" ENTÃO
    Req = SOMA(lista_resistores)
SENÃO
    Req = 1 / SOMA(1 / r PARA CADA r EM lista_resistores)
FIM_SE
I = tensao_v / Req
P = tensao_v * I
ESCREVA Req, I, P
FIM

```

Código Python:

```

None
def calcular_circuito():
    n = int(input("Quantos resistores? "))
    resistores = [float(input(f"R{i+1} (Ohm): ")) for i in range(n)]
    tipo = input("Associação (serie/paralelo): ").lower()
    v = float(input("Tensão de alimentação (V): "))

    if tipo == "serie":
        req = sum(resistores)
    else:
        req = 1 / sum(1/r for r in resistores)

    corrente = v / req
    potencia = v * corrente
    print(f"Req: {req:.2f} Ohm, Corrente: {corrente:.4f} A, Potência: {potencia:.2f} W")

calcular_circuito()

```

Exercício 2: Controle de Qualidade de Peças (Mecânica)

Enunciado: Um engenheiro mecânico precisa validar o diâmetro de um lote de eixos. Crie um algoritmo que leia o diâmetro nominal, a tolerância permitida e os diâmetros de N peças. O programa deve contar quantas foram aprovadas e calcular o desvio médio absoluto.

Pseudocódigo:

None

```
ALGORITMO Controle_Qualidade
INÍCIO
    LEIA nominal, tolerancia, n_pecas
    aprovadas = 0
    soma_desvios = 0
    PARA cada_peca DE 1 ATÉ n_pecas:
        LEIA medida
        desvio = medida - nominal
        soma_desvios = soma_desvios + ABS(desvio)
        SE ABS(desvio) <= tolerancia ENTÃO
            aprovadas = aprovadas + 1
        FIM_SE
    FIM_PARA
    ESCREVA aprovadas, soma_desvios / n_pecas
FIM
```

Código Python:

None

```
def controle_qualidade():
    nominal = float(input("Diâmetro nominal (mm): "))
    tol = float(input("Tolerância (+/- mm): "))
    n = int(input("Quantidade de peças: "))
    aprovadas = 0
    soma_desvios = 0

    for i in range(n):
        medida = float(input(f"Peça {i+1}: "))
        desvio = medida - nominal
        soma_desvios += abs(desvio)
        if abs(desvio) <= tol:
            aprovadas += 1

    print(f"Aprovadas: {aprovadas} de {n}")
    print(f"Desvio médio: {soma_desvios/n:.4f} mm")

controle_qualidade()
```

Exercício 3: Tabela de Comprimento de Onda (Telecomunicações)

Enunciado: Gere uma tabela que converta uma faixa de frequências (em MHz) para comprimentos de onda (λ), dado que $\lambda = c/f$, onde $c = 3 \times 10^8 \text{ m/s}$. O usuário deve informar a frequência inicial, final e o passo.

Pseudocódigo:

None

ALGORITMO Tabela_Lambda

INÍCIO

```
    LEIA f_inicial, f_final, passo (em MHz)
    c = 3e8
    ESCRIVA "Frequência (MHz) | Comprimento (m)"
    PARA f DE f_inicial ATÉ f_final COM passo:
        f_hz = f * 1e6
        lamb = c / f_hz
        ESCRIVA f, lamb
    FIM_PARA
```

FIM

Código Python:

None

```
def tabela_lambda():
    f_ini = int(input("Frequência inicial (MHz): "))
    f_fim = int(input("Frequência final (MHz): "))
    passo = int(input("Passo (MHz): "))
    c = 3e8

    print(f"{'Freq (MHz)':>10} | {'Lambda (m)':>10}")
    for f in range(f_ini, f_fim + 1, passo):
        l_onda = c / (f * 1e6)
        print(f"{f:10d} | {l_onda:10.4f}")

tabela_lambda()
```

Exercício 4: Simulação de Descarga de Capacitor (Elétrica)

Enunciado: Implemente um algoritmo que simule a queda de tensão em um capacitor usando a fórmula $V(t) = V_0 \cdot e^{-t/RC}$. O programa deve exibir o tempo e a tensão a cada 0.1s até que a tensão caia abaixo de 1% do valor inicial (V_0).

Pseudocódigo:

None

ALGORITMO Descarga_Capacitor

INÍCIO

```
    LEIA V0, R, C
    t = 0
    V = V0
    ENQUANTO V > (V0 * 0.01):
        ESCRIVA t, V
        t = t + 0.1
```

```
        V = V0 * EXP(-t / (R * C))
FIM_ENQUANTO
FIM
```

Código Python:

```
None
import math

def descarga_capacitor():
    v0 = float(input("Tensão inicial (V): "))
    r = float(input("Resistência (Ohm): "))
    c = float(input("Capacitância (F): "))
    t = 0.0
    v = v0

    print(f"{'Tempo (s)':>8} | {'Tensão (V)':>10}")
    while v > (v0 * 0.01):
        print(f"{t:8.2f} | {v:10.3f}")
        t += 0.1
        v = v0 * math.exp(-t / (r * c))

descarga_capacitor()
```

Exercício 5: Ensaio de Fadiga de Materiais (Mecânica)

Enunciado: Simule a degradação da resistência de um material submetido a ciclos de fadiga. A cada ciclo, a resistência diminui 0,01%. O programa deve ler a resistência inicial e calcular quantos ciclos são necessários para que ela atinja 80% do valor original.

Pseudocódigo:

```
None
ALGORITMO Fadiga_Material
INÍCIO
    LEIA r_inicial
    r_atual = r_inicial
    limite = r_inicial * 0.8
    ciclos = 0
    ENQUANTO r_atual > limite:
        r_atual = r_atual * 0.9999
        ciclos = ciclos + 1
```

```
FIM_ENQUANTO
ESCREVA ciclos
FIM
```

Código Python:

```
None
def simular_fadiga():
    r_ini = float(input("Resistência inicial do material: "))
    r_atual = r_ini
    limite = r_ini * 0.8
    ciclos = 0

    while r_atual > limite:
        r_atual *= 0.9999
        ciclos += 1

    print(f"O material falhou após {ciclos} ciclos.")

simular_fadiga()
```

Exercício 6: Atenuação em Linha de Transmissão (Telecomunicações)

Enunciado: Uma linha de transmissão perde 0,5% de tensão a cada quilômetro. Escreva um programa que receba a tensão inicial e a distância total, exibindo a tensão km a km e emitindo um alerta se a tensão cair abaixo de 90% da inicial.

Pseudocódigo:

```
None
ALGORITMO Atenuacao_Linha
INÍCIO
    LEIA v_inicial, distancia_total
    v_atual = v_inicial
    PARA km DE 1 ATÉ distancia_total:
        v_atual = v_atual * 0.995
        ESCREVA km, v_atual
        SE v_atual < (v_inicial * 0.9) ENTÃO
            ESCREVA "ALERTA: Tensão abaixo de 90%!"
    FIM_SE
FIM_PARA
FIM
```

Código Python:

None

```
def atenuacao_transmissao():
    v_ini = float(input("Tensão de entrada (V): "))
    dist = int(input("Distância total (km): "))
    v_atual = v_ini

    for km in range(1, dist + 1):
        v_atual *= 0.995
        print(f"Km {km}: {v_atual:.2f} V", end="")
        if v_atual < v_ini * 0.9:
            print(" - ALERTA: Subtensão!")
        else:
            print()

atenuacao_transmissao()
```

Exercício 7: Cálculo de Ganho de Antena (Telecomunicações)

Enunciado: Crie um algoritmo que calcule o ganho aproximado de uma antena Yagi de N elementos, usando a fórmula $G = 10 \cdot \log_{10}(0.8 \cdot n)$. O programa deve gerar uma tabela para antenas de 2 a 20 elementos.

Pseudocódigo:

None

```
ALGORITMO Ganho_Yagi
INÍCIO
    PARA n DE 2 ATÉ 20:
        ganho = 10 * LOG10(0.8 * n)
        ESCREVA n, ganho
    FIM_PARA
FIM
```

Código Python:

None

```
import math

def ganho_antena():
    print(f"{'Elementos':>10} | {'Ganho (dBi)':>10}")
    for n in range(2, 21):
        ganho = 10 * math.log10(0.8 * n)
        print(f"{n:10d} | {ganho:10.2f}")

ganho_antena()
```

Exercício 8: Cálculo de Torque em Eixos (Mecânica)

Enunciado: Escreva uma função que receba a potência de um motor (P em Watts) e a rotação (n em RPM) e retorne o torque (T) em N.m, usando $T=P/\omega$, onde $\omega=2\pi n/60$. O programa deve validar se a rotação é positiva.

Pseudocódigo:

None

ALGORITMO Calculo_Torque

INÍCIO

LEIA potencia, rpm

SE rpm <= 0 **ENTÃO**

ESCREVA "Erro: Rotação deve ser positiva"

SENÃO

 omega = (2 * 3.14159 * rpm) / 60

 torque = potencia / omega

ESCREVA torque

FIM_SE

FIM

Código Python:

None

```
import math
```

```
def calcular_torque(p, n):
```

```
    if n <= 0:
```

```
        return None
```

```
    omega = (2 * math.pi * n) / 60
```

```
    return p / omega
```

```
pot = float(input("Potência (W): "))
```

```
rot = float(input("Rotação (RPM): "))
```

```
res = calcular_torque(pot, rot)
```

```
if res:
```

```
    print(f"Torque: {res:.2f} N.m")
```

```
else:
```

```
    print("Entrada inválida.")
```

Exercício 9: Estatísticas de Sensores (Engenharia Geral)

Enunciado: Um sistema SCADA coleta 10 leituras de um sensor de pressão. Desenvolva um algoritmo que armazene as leituras em uma lista e exiba a média, o valor máximo e o valor mínimo.

Pseudocódigo:

None

ALGORITMO Stats_Sensor

INÍCIO

```
    lista_leituras = []
    PARA i DE 1 ATÉ 10:
        LEIA p
        ADICIONE p À lista_leituras
    FIM_PARA
    ESCREVA MÉDIA(lista_leituras), MÁXIMO(lista_leituras),
MÍNIMO(lista_leituras)
FIM
```

Código Python:

None

```
def estatisticas_sensor():
    leituras = []
    for i in range(10):
        p = float(input(f"Leitura de pressão {i+1} (bar): "))
        leituras.append(p)

    print(f"Média: {sum(leituras)/10:.2f} bar")
    print(f"Máxima: {max(leituras):.2f} bar")
    print(f"Mínima: {min(leituras):.2f} bar")

estatisticas_sensor()
```

Exercício 10: Dimensionamento de Condutores (Elétrica)

Enunciado: Implemente um programa que receba a corrente de projeto (I) e sugira a bitola do cabo baseada na seguinte regra simplificada: $I \leq 10A \rightarrow 1.5\text{mm}^2$; $I \leq 16A \rightarrow 2.5\text{mm}^2$; $I \leq 25A \rightarrow 4.0\text{mm}^2$; $I \leq 35A \rightarrow 6.0\text{mm}^2$. Para valores maiores, exibir "Consultar tabelas da NBR 5410".

Pseudocódigo:

None

ALGORITMO Bitola_Cabo

INÍCIO

```
    LEIA corrente
    SE corrente <= 10 ENTÃO bitola = "1.5 mm2"
    SENÃO SE corrente <= 16 ENTÃO bitola = "2.5 mm2"
    SENÃO SE corrente <= 25 ENTÃO bitola = "4.0 mm2"
    SENÃO SE corrente <= 35 ENTÃO bitola = "6.0 mm2"
    SENÃO bitola = "Consultar NBR 5410"
    FIM_SE
```

```
ESCREVA bitola
FIM
```

Código Python:

```
None
def dimensionar_cabo():
    i = float(input("Corrente do circuito (A): "))
    if i <= 10:
        bitola = "1.5 mm2"
    elif i <= 16:
        bitola = "2.5 mm2"
    elif i <= 25:
        bitola = "4.0 mm2"
    elif i <= 35:
        bitola = "6.0 mm2"
    else:
        bitola = "Consultar NBR 5410"

    print(f"Bitola sugerida: {bitola}")

dimensionar_cabo()
```

Lista de Respostas (Gabarito Lógico)

1. Resistência Equivalente: Usa sum() para série e soma de inversos para paralelo.
2. Qualidade: Aprovada se abs(medida - nominal) <= tolerancia.
3. Telecom: Aplica conversão MHz para Hz (10^6) antes de dividir c.
4. Capacitor: Loop while para condição dinâmica de tensão.
5. Fadiga: Loop while com redução geométrica (multiplicação por 0,9999).
6. Linha: Uso de range() e verificação de limite crítico km a km.
7. Antena: Aplicação de math.log10 em uma faixa definida.
8. Torque: Conversão de RPM para rad/s é essencial antes do cálculo final.
9. Stats: Funções nativas max() e min() em listas facilitam o trabalho.
10. Bitola: Estrutura if/elif/else para faixas de decisão técnica.

OBSERVAÇÕES DAS LLMs

- **ChatGPT:**

Ao fazer o pedido de criar 10 exercícios sobre algoritmos, o NotebookLM entregou todas as questões completas, conforme solicitado. Elaborou todos os exercícios contextualizados para problemas de Engenharia Mecânica, Elétrica e de Telecomunicação com códigos em pseudocódigo em linguagem algorítmica em português e códigos em Python. Os enunciados são claros e fácil entendimento, porém os exercícios não são variados, sempre focando em códigos parecidos.

Ao analisar os pseudocódigos em linguagem algorítmica em português, observa-se que a LLM formulou pseudocódigos de fácil entendimento, que poderão auxiliar no momento da escrita em Python e tem coerência com o problema apresentado. Os códigos em Python foram testados e funcionam normalmente.

Notou-se que o ChatGPT não formulou questões de nível intermediário. Mesmo que as questões tivessem enunciados contextualizados e os dois tipos de linguagem exigida, ele não formulou questões mais complexas, limitando-se a atividades com enunciados de apenas uma frase e baixo grau de dificuldade.

No gabarito, o ChatGPT mostrou as fórmulas já implementadas nos exercícios anteriores, sem ter um real gabarito.

Observação: gera exercícios menos complexos e sem criatividade(comparado com o Gemini e o NotebookLM), resoluções coerentes e códigos legíveis.

Nota: 7/10

- **Gemini**

Ao fazer o pedido de criar 10 exercícios sobre algoritmos, o Gemini não entregou todas as questões completas. A partir da questão quatro (4), a LLM não apresentou as resoluções em pseudocódigo em linguagem algorítmica em português exigidas, apenas códigos em Python. Entretanto, elaborou todos os exercícios com enunciados claros e contextualizados para problemas de Engenharia Mecânica, Elétrica e de Telecomunicação.

Os pseudocódigos em linguagem algorítmica em português nos exercícios 1, 2 e 3 são de fácil entendimento, podendo auxiliar no momento da escrita em Python e tem coerência com o problema apresentado. Os códigos em Python foram testados e funcionam normalmente.

Como proposto, o Gemini formulou questões de nível intermediário. Nota-se que as atividades têm um grau de complexidade maior e estimulam o crescimento cognitivo.

No gabarito, o Gemini estipula valores para cada incógnita e expôs a resposta de maneira resumida.

Observação: gera exercícios complexos, resoluções coerentes e códigos legíveis, criativo, não realiza os prompts completos.

Nota: 7/10

- **NotebookLM**

Ao fazer o pedido de criar 10 exercícios sobre algoritmos, o NotebookLM entregou todas as questões completas, conforme solicitado. Elaborou todos os exercícios contextualizados para problemas de Engenharia Mecânica, Elétrica e de Telecomunicação com códigos em pseudocódigo em linguagem algorítmica em português e códigos em Python. Os enunciados são claros, porém não especificam as fórmulas que deverão ser usadas na criação dos códigos. Além disso, os exercícios são bastante variados, o que ajuda no aprendizado.

Ao analisar os pseudocódigos em linguagem algorítmica em português, observa-se que a LLM formulou pseudocódigos de fácil entendimento, que poderão auxiliar no momento da escrita em Python e tem coerência com o problema apresentado. Os códigos em Python foram testados e funcionam normalmente.

Como proposto, o NotebookLM formulou questões de nível intermediário. Nota-se que as atividades têm um grau de complexidade maior e estimulam o crescimento cognitivo.

No gabarito, o NotebookLM orientou, com poucas palavras, o que poderia ser feito em cada questão.

Observação: gera exercícios complexos, resoluções coerentes e códigos legíveis, criativo, realiza exercícios personalizados a partir do material enviado.

Nota: 9/10

Entre as 3 LLM (ChatGPT, Gemini e NotebookLM) o melhor para elaboração de problemas e estudos é o NotebookLM, seguido do Gemini e ChatGPT.

NotebookLM > Gemini > ChatGPT